

DAY 9: PYTHON MODULES, PACKAGES & CODE ORGANIZATION

Data Science 60-Day Learning Journey

Student: Muhammad Ahmad

Status: Day 9 Completed

Focus: Modules, imports, built-in modules, custom modules, packages, reusable code, and modular Data Science projects.

1. Learning Objectives

Understand Python modules and packages, import built-in and custom modules, use aliases, import specific functions, create reusable Python files, understand `__name__ == '__main__'`, and organize a small Data Science project into separate files.

2. What Is a Module?

A module is a Python file containing reusable code such as functions, variables, classes, or other statements. Instead of repeating code, you can place it in a module and import it.

```
# calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# main.py
import calculator

print(calculator.add(10, 5))
print(calculator.subtract(10, 5))
```

Output: 15 and 5. The main program reuses functions from another file.

3. Why Use Modules?

Modules help reuse code, avoid duplication, organize projects, separate responsibilities, simplify testing, and make larger programs easier to maintain.

```
data_science_project/
■■■ main.py
■■■ data_loader.py
■■■ data_cleaning.py
■■■ analysis.py
■■■ visualization.py
■■■ model.py
```

4. Importing Modules

```
import math
print(math.sqrt(25))

import random
print(random.randint(1, 10))
```

The import statement loads a module so its functions and other objects can be used.

5. The math Module

```
import math

print(math.sqrt(16))
print(math.pow(2, 3))
print(math.ceil(4.2))
```

```
print(math.floor(4.9))
print(math.pi)
```

Common functions include `sqrt()`, `pow()`, `ceil()`, and `floor()`. `math.pi` provides the mathematical constant `pi`.

6. Import Specific Functions

```
from math import sqrt
print(sqrt(25))

from math import pi, sqrt
radius = 5
area = pi * radius ** 2
print(area)
```

7. Import Aliases

```
import math as m
print(m.sqrt(25))

# Standard Data Science aliases
import numpy as np
import pandas as pd
```

Aliases make code shorter. `np` and `pd` are standard conventions in Data Science.

8. random Module

```
import random

print(random.randint(1, 100))

students = ["Ahmad", "Ali", "Usman", "Hamza"]
print(random.choice(students))

print(random.random())
```

`randint()` creates a random integer, `choice()` selects an item, and `random()` creates a random float between 0 and 1.

9. datetime Module

```
from datetime import datetime

now = datetime.now()

print(now)
print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second:", now.second)
```

10. os Module

```
import os

print(os.getcwd())

if os.path.exists("students.json"):
    print("File exists")
else:
    print("File does not exist")

os.makedirs("data", exist_ok=True)
```

`os` is useful for working with folders, paths, and the operating system.

11. sys Module

```
import sys
print(sys.version)
```

sys provides information and functionality related to the Python interpreter.

12. Creating Your Own Module

```
# student_utils.py

def calculate_average(marks):
    return sum(marks) / len(marks)

def find_highest(marks):
    return max(marks)

# main.py

import student_utils

marks = [80, 75, 90, 85]

print("Average:", student_utils.calculate_average(marks))
print("Highest:", student_utils.find_highest(marks))
```

13. Import Specific Functions from Your Module

```
from student_utils import calculate_average

marks = [80, 75, 90]
print(calculate_average(marks))
```

14. Module Aliases

```
import student_utils as su

marks = [80, 75, 90]
print(su.calculate_average(marks))
```

15. `__name__ == '__main__'`

Every Python module has `__name__`. When a file is run directly, its value is `'__main__'`. When imported, its value is the module name. This lets you keep reusable functions separate from code that should execute only when the file is run directly.

```
# calculator.py

def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Calculator running directly")
    print(add(10, 5))
```

16. What Is a Package?

A package is a folder that groups related Python modules.

```
student_project/
■■■ main.py
■■■ utils/
    ■■■ __init__.py
    ■■■ students.py
    ■■■ calculations.py

# utils/calculations.py
def average(numbers):
    return sum(numbers) / len(numbers)

# main.py
from utils.calculations import average
print(average([80, 70, 90]))
```

17. Modules in Data Science

```
student_analysis/
```

```

■■■ main.py
■■■ data_loader.py
■■■ data_cleaning.py
■■■ statistics.py
■■■ visualization.py
■■■ model.py

# data_loader.py
import pandas as pd

def load_data(filename):
    return pd.read_csv(filename)

# data_cleaning.py
def remove_missing_values(data):
    return data.dropna()

# main.py
from data_loader import load_data
from data_cleaning import remove_missing_values

data = load_data("students.csv")
clean_data = remove_missing_values(data)
print(clean_data)

```

This structure becomes especially useful with NumPy, Pandas, data cleaning, visualization, and Machine Learning.

18. Practice Tasks with Complete Solutions

Task 1: Create *calculator.py* with addition, subtraction, multiplication, and division functions. Import the module into *main.py* and use all four functions.

Solution:

```

# calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b

# main.py
import calculator

print("Addition:", calculator.add(10, 5))
print("Subtraction:", calculator.subtract(10, 5))
print("Multiplication:", calculator.multiply(10, 5))
print("Division:", calculator.divide(10, 5))

```

Task 2: Use *math* to calculate square root, power, ceiling, and floor.

Solution:

```

import math

print("Square root:", math.sqrt(16))
print("Power:", math.pow(2, 3))
print("Ceiling:", math.ceil(4.2))
print("Floor:", math.floor(4.9))

```

Task 3: Use *random* to randomly select one student from a list of five students.

Solution:

```

import random

students = ["Ahmad", "Ali", "Usman", "Hamza", "Bilal"]
selected_student = random.choice(students)
print("Selected student:", selected_student)

```

Task 4: Use *datetime* to display the current year, month, day, hour, minute, and second.

Solution:

```

from datetime import datetime

now = datetime.now()

print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second:", now.second)

```

Task 5: Use os to check whether students.json exists and display an appropriate message.

Solution:

```

import os

if os.path.exists("students.json"):
    print("File exists")
else:
    print("File does not exist")

```

Task 6: Create student_utils.py with functions for average marks and highest marks, then import and use both functions from another file.

Solution:

```

# student_utils.py
def calculate_average(marks):
    return sum(marks) / len(marks)

def find_highest(marks):
    return max(marks)

# main.py
import student_utils

marks = [80, 75, 90, 85]

print("Average:", student_utils.calculate_average(marks))
print("Highest:", student_utils.find_highest(marks))

```

Task 7: Create a module with a CGPA calculation function. Import it into another program and test multiple values.

Solution:

```

# cgpa_utils.py
def calculate_cgpa(total_grade_points, total_credit_hours):
    if total_credit_hours == 0:
        return 0
    return total_grade_points / total_credit_hours

# main.py
from cgpa_utils import calculate_cgpa

print(calculate_cgpa(32.3, 10))
print(calculate_cgpa(36.0, 10))
print(calculate_cgpa(28.5, 10))

```

Task 8: Create a student_tools package containing marks.py and students.py. Import functions from both modules into main.py.

Solution:

```

# student_tools/marks.py
def average_marks(marks):
    return sum(marks) / len(marks)

def highest_marks(marks):
    return max(marks)

# student_tools/students.py
def student_name(student):
    return student.get("name", "Unknown")

def is_high_performer(student):
    return student.get("cgpa", 0) >= 3.0

# main.py

```

```

from student_tools.marks import average_marks, highest_marks
from student_tools.students import student_name, is_high_performer

marks = [80, 75, 90]
student = {"name": "Ahmad", "cgpa": 3.23}

print(average_marks(marks))
print(highest_marks(marks))
print(student_name(student))
print(is_high_performer(student))

```

Task 9: Create a program using `if __name__ == '__main__':`. The main section should run when the file is executed directly but not when imported.

Solution:

```

# calculator.py
def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Calculator running directly")
    print("Result:", add(10, 5))

```

When another file imports calculator.py, add() is reusable while the main section does not execute automatically.

Task 10: Create a modular Student Data Analyzer using separate modules for data loading, calculations, filtering, and the main program. Use JSON, functions, lists, dictionaries, filtering, and exception handling.

Solution:

```

# data_loader.py
import json

def load_students(filename):
    try:
        with open(filename, "r") as file:
            return json.load(file)
    except FileNotFoundError:
        print("Student file not found")
        return []
    except json.JSONDecodeError:
        print("Invalid JSON data")
        return []

# calculations.py
def average_marks(students):
    if not students:
        return 0
    return sum(s["marks"] for s in students) / len(students)

def highest_cgpa(students):
    if not students:
        return 0
    return max(s["cgpa"] for s in students)

# filters.py
def high_cgpa_students(students, minimum_cgpa=3.0):
    return [s for s in students if s.get("cgpa", 0) >= minimum_cgpa]

# main.py
from data_loader import load_students
from calculations import average_marks, highest_cgpa
from filters import high_cgpa_students

students = load_students("students.json")

if students:
    for student in students:
        print(student)

    print("Average Marks:", average_marks(students))
    print("Highest CGPA:", highest_cgpa(students))

    print("Students with CGPA >= 3.0:")
    for student in high_cgpa_students(students):
        print(student["name"])
else:
    print("No student data available")

```

19. Mini Project: Modular Student Data Analyzer

Convert the Day 8 analyzer into a multi-file project. The goal is to separate data loading, calculations, filtering, and program execution.

```
student_analyzer/  
■■■ main.py  
■■■ data_loader.py  
■■■ calculations.py  
■■■ filters.py
```

data_loader.py reads students.json and handles file and JSON errors. calculations.py handles average marks and highest CGPA. filters.py handles CGPA-based filtering. main.py imports the functions and displays the analysis. The complete solution is Task 10.

20. Common Mistakes

Avoid putting the entire project in one huge file, repeating the same functions, using unclear module names, importing unnecessary objects, ignoring empty data, dividing by zero, or allowing test code to run automatically when a reusable module is imported.

21. Quick Revision

```
Module  
→ Reusable Python file  
  
import  
→ Load a module  
  
from ... import ...  
→ Import specific objects  
  
as  
→ Create an alias  
  
Package  
→ Folder containing related modules  
  
__name__ == "__main__"  
→ Run code only when the file is executed directly
```

22. Data Science Connection

Professional Data Science projects use modular code because data loading, cleaning, analysis, visualization, feature engineering, and model training can become large. Separating responsibilities makes code easier to test, debug, reuse, and maintain.

```
Data Loading  
↓  
Data Cleaning  
↓  
Data Analysis  
↓  
Visualization  
↓  
Feature Engineering  
↓  
Machine Learning  
↓  
Evaluation
```

23. Day 9 Progression

```
Python Basics  
↓  
Conditions + Loops  
↓  
Functions  
↓  
Lists  
↓
```

Tuples + Sets + Dictionaries
↓
File Handling
↓
CSV + JSON
↓
Exception Handling + Debugging
↓
Modules + Packages
↓
NumPy
↓
Pandas
↓
Data Cleaning
↓
Visualization
↓
Statistics
↓
Machine Learning

24. Final Day 9 Checklist

- I understand modules.
- I can import built-in modules.
- I can import specific functions.
- I understand aliases such as np and pd.
- I can create my own module.
- I can create a package.
- I understand `__name__ == '__main__'`.
- I can organize a multi-file project.
- I can separate Data Science responsibilities into modules.
- I completed the 10 practice tasks.
- I understand the Modular Student Data Analyzer.

DAY 9 COMPLETED: Python Modules, Packages & Code Organization